

Оглавление

1	Аналитический раздел	5
1.1	Анализ предметной области	5
1.2	Сравнительный анализ существующих систем	5
1.3	Требования к проектируемой системе	6
1.3.1	Функциональные требования	6
1.3.2	Нефункциональные требования	6
1.3.3	Ограничения	7
1.4	Метрики эффективности и правила их расчёта	7
1.5	Сущности предметной области, роли пользователей и варианты использования .	8
1.5.1	Сущности и связи	8
1.5.2	Роли пользователей и варианты использования	9
1.6	Выбор модели данных	10
2	Конструкторский раздел	13
2.1	Логическое проектирование базы данных	13
2.1.1	Соответствие нормальным формам	18
2.1.2	Проектирование представлений	18
2.1.3	Проектирование индексов	19
2.2	Проектирование ограничений целостности	19
2.3	Проектирование алгоритмов обработки данных	20
2.3.1	Алгоритмы расчёта метрик игроков	20
2.3.2	Алгоритм агрегирования командной статистики	20
2.3.3	Алгоритм массового обновления агрегатов	21
2.4	Проектирование ролевой модели доступа	21
2.5	Проектирование программного обеспечения и кэширования	22
2.5.1	Архитектура приложения	22
2.5.2	Спецификация REST API	22
2.5.3	Кэширование результатов запросов	24
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	26

1 Аналитический раздел

1.1 Анализ предметной области

Национальная баскетбольная ассоциация (NBA) является источником одного из наиболее детализированных массивов спортивной статистики среди крупнейших мировых профессиональных лиг [1]. Для каждого матча фиксируются итоговый счёт, составы команд и числовые показатели по каждому игроку: очки, подборы, передачи, блок-шоты, перехваты и иные. На этой основе можно строить рейтинги, сопоставлять сезоны и выявлять тенденции развития игры.

В современной баскетбольной аналитике для комплексной оценки результативности применяются продвинутое метрики, стандартизированные в работах ведущих спортивных статистиков [2]. Расчёт таких показателей как истинный процент бросков TS%, эффективный процент с игры eFG%, доля владений в атаке USG%, рейтинг эффективности игрока PER и показатель вклада игрока BPM требует хранения непротиворечивых первичных данных по матчам, их агрегирования по сезонам и применения фиксированных правил вычисления.

Обработка спортивной статистики требует строгого контроля над целостностью данных: сумма очков игроков должна совпадать с командным счётом, агрегаты по сезонам должны вычисляться из одних и тех же первичных фактов, а параллельный доступ не должен нарушать консистентность записей. Это определяет необходимость специализированного хранилища с поддержкой транзакционной целостности и декларативной валидации бизнес-правил.

1.2 Сравнительный анализ существующих систем

Перед проектированием собственного решения проведём анализ функциональных возможностей наиболее распространённых публичных систем сбора и анализа статистики NBA: официального портала NBA Stats [1], справочной платформы Basketball-Reference [4] и аналитического раздела ESPN [5].

Таблица 1.1 — Сравнительный анализ существующих систем по функциональным возможностям

Критерий	NBA.com Stats	Basketball-Reference	ESPN Stats
Просмотр статистики матчей	+	+	+
Фильтрация по сезону, позиции, команде	частично	+	частично
Расчёт TS%, eFG%, PER, BPM	+(формулы скрыты)	+(формулы открыты [7, 8])	частично
Произвольные аналитические запросы	–	–	–
Сравнение игроков	частично	–	+
Ролевое разграничение доступа	–	–	–
Локальное хранение, воспроизводимость расчётов	–	–	–

Анализ показывает, что NBA.com Stats и ESPN предоставляют данные через готовый пользовательский интерфейс с непрозрачными формулами расчёта метрик. Basketball-Reference публикует описание своих формул [7, 8], однако не предоставляет открытого API для произвольных аналитических запросов. Ни одна из систем не поддерживает ролевого разграничения доступа и воспроизводимого локального хранения данных. Выявленные ограничения определяют требования к проектируемой системе, сформулированные в следующем разделе.

1.3 Требования к проектируемой системе

1.3.1 Функциональные требования

- 1) хранение справочных данных: игровые позиции, сезоны, клубы NBA;
- 2) хранение профилей игроков: персональные и антропометрические данные, позиция, сведения о драфте;
- 3) хранение данных о матчах: сезон, команды-участники, дата, итоговый счёт, статус;
- 4) хранение детализированной статистики игрока в матче: очки, подборы, передачи, броски с игры и штрафные, потери, фолы, время на площадке;
- 5) хранение агрегированной статистики игрока за сезон: средние значения, процентные показатели, расчётные метрики;
- 6) хранение истории выступлений игрока за клубы по сезонам;
- 7) расчёт командной статистики в рамках матча и сезона на основе индивидуальных показателей игроков;
- 8) автоматизированный расчёт метрик эффективности (eFG%, TS%, USG%, PER, BPM) на основе первичных фактов матчей;
- 9) предоставление сводных рейтингов игроков и турнирных таблиц команд.

1.3.2 Нефункциональные требования

- 1) производительность: время отклика на аналитические запросы чтения (агрегация за сезон) не должно превышать 200 мс;
- 2) объём данных: система должна выдерживать нагрузку не менее 100 000 записей в таблице фактов (соответствует статистике за 5 регулярных сезонов);
- 3) целостность: база данных должна соответствовать требованиям ACID, обеспечивая ссылочную и логическую целостность на уровне СУБД;
- 4) безопасность: ролевое разграничение доступа (администратор, аналитик, читатель), реализованное средствами СУБД на уровне таблиц и представлений;
- 5) масштабируемость чтения: система должна обеспечивать кэширование результатов повторяющихся аналитических запросов для снижения нагрузки на дисковую подсистему при конкурентном доступе.

1.3.3 Ограничения

- 1) глубина хранения данных ограничена 5 регулярными сезонами (с 2019/20 по 2023/24);
- 2) пользовательский интерфейс предназначен для просмотра и анализа данных, а не для ввода первичной статистики в режиме реального времени.

1.4 Метрики эффективности и правила их расчёта

Обозначения: PTS — очки; FGM, FGA — попадания и попытки с игры; FG3M — трёхочковые; FTM, FTA — штрафные; TOV — потери; TRB — подборы; AST — передачи; MP — минуты. Префикс Tm обозначает командный агрегат по матчам с участием игрока: TmFGA, TmFTA, TmTOV; TmMP — суммарное количество минут, отыгранных всеми игроками команды в матчах, учитываемых при расчёте показателя данного игрока.

Командная статистика не выделяется в отдельную сущность: она является производной от индивидуальных показателей и вычисляется динамически суммированием по всем игрокам команды в конкретном матче.

Эффективный процент с игры, eFG%. Метрика корректирует ценность трёхочкового броска. Согласно методологии NBA [9], формула имеет вид:

$$eFG\% = \frac{FGM + 0,5 \cdot FG3M}{FGA} \quad (1.1)$$

Истинный процент бросков, TS%. Показатель объединяет броски с игры и штрафные броски в единую метрику эффективности. Множитель 0,44 в знаменателе является эмпирически выведенным коэффициентом Дина Оливера, учитывающим долю штрафных бросков, завершающих владение [10]:

$$TS\% = \frac{PTS}{2 \cdot (FGA + 0,44 \cdot FTA)} \quad (1.2)$$

Доля владений в атаке, USG%. Оценивает процент командных владений, которые завершает конкретный игрок, находясь на площадке [2]. В базе данных показатель хранится как доля [0; 1]; в интерфейсе отображается в процентах ($\times 100$). Формула:

$$USG = \frac{(FGA + 0,44 \cdot FTA + TOV) \cdot (TmMP/5)}{MP \cdot (TmFGA + 0,44 \cdot TmFTA + TmTOV)} \quad (1.3)$$

Деление TmMP на 5 нормирует командное время с учётом того, что на площадке одновременно находятся пять игроков.

Рейтинг эффективности игрока, PER. Оригинальная формула разработана Джоном Холлинджером [6]. В настоящей работе применяется её упрощённая версия без позиционных поправок, методология которой описана на портале Basketball-Reference [7]. Подборы TRB = REB_OFF + REB_DEF. Ненормализованный вклад приводится к шкале «за 48 минут», затем

масштабируется к среднему по лиге, равному 15:

$$uPER_i = \left[PTS + TRB + AST + STL + BLK - TOV - PF - (FGA - FGM) - 0,44 \cdot (FTA - FTM) \right] \cdot \frac{48}{MP_i} \quad (1.4)$$

$$lg_aPER = AVG(uPER_i) \quad \text{по игрокам с } MP_i > 100 \quad (1.5)$$

$$PER_i = uPER_i \cdot \frac{15}{lg_aPER} \quad (1.6)$$

Минимальный порог $MP_i \geq 50$ минут за сезон; для расчёта lg_aPER учитываются игроки с $MP_i > 100$ минут.

Вклад игрока (Box Plus/Minus, BPM). Метрика разработана Дэниелом Майерсом [8]. В системе используется классическая линейная аппроксимация первой версии алгоритма (BPM 1.0) с нормировкой показателей на 100 минут ($X_{100} = X/MP \cdot 100$) и учётом перехватов, блоков и фолов. Итоговая формула:

$$raw_i = 0,123 \cdot PTS_{100} + 0,234 \cdot TRB_{100} + 0,689 \cdot AST_{100} + 0,445 \cdot STL_{100} + 0,407 \cdot BLK_{100} - 0,605 \cdot TOV_{100} - 0,086 \cdot PF_{100} \quad (1.7)$$

$$BPM_i = raw_i - AVG(raw_i) \quad \text{по игрокам с } MP_i > 100 \quad (1.8)$$

Среднее BPM по лиге после вычитания среднего равно 0. Минимальный порог для расчёта показателя игрока — $MP_i \geq 50$ минут за сезон.

1.5 Сущности предметной области, роли пользователей и варианты использования

1.5.1 Сущности и связи

По результатам анализа предметной области выделены восемь основных сущностей.

- 1) позиция — справочник игровых позиций в номенклатуре NBA;
- 2) сезон — игровой год, его календарные границы и тип турнира;
- 3) команда — клуб NBA: конференция, дивизион, арена;
- 4) игрок — персональные и антропометрические данные, позиция, сведения о драфте;
- 5) матч — сезон, команды-участники, дата, счёт, статус;
- 6) статистика игрока в матче — числовые показатели конкретного игрока в конкретной игре; основная таблица фактов предметной области;
- 7) статистика игрока за сезон — агрегаты и расчётные метрики по тройке «игрок – сезон – команда»;
- 8) история выступлений — связь «игрок – команда – сезон» с датами и типом контракта.

Связь «многие-ко-многим» между игроками и матчами реализуется через сущность «Статистика игрока в матче»: один игрок участвует во многих матчах, в одном матче фикс-

сируется статистика многих игроков [3, 11].

Связь «Проходит в» на ER-диаграмме является тернарной: она одновременно связывает сущности «Матч», «Команда» и «Сезон», поскольку матч всегда привязан к конкретному сезону и двум командам-участницам [11].

Концептуальная ER-диаграмма в нотации Чена приведена на рисунке 1.1; на ней отражены сущности предметной области, бинарные и тернарные связи.

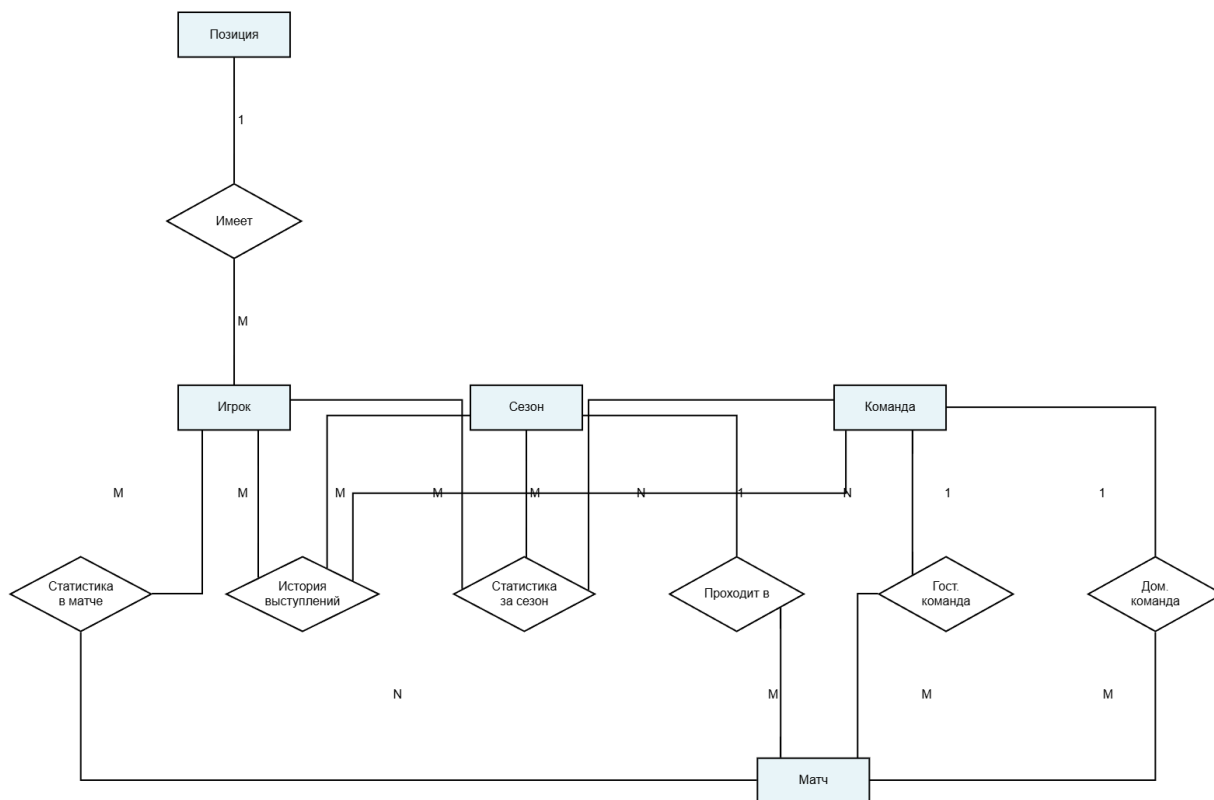


Рисунок 1.1 — ER-диаграмма в нотации Чена

1.5.2 Роли пользователей и варианты использования

Система предусматривает три роли пользователей.

- 1) администратор — полное управление структурой и содержимым хранилища: создание и изменение схемы, загрузка и сопровождение данных, резервное копирование;
- 2) аналитик — чтение данных и запуск расчётных процедур для получения производных показателей;
- 3) читатель — доступ к агрегированным данным через представления; изменение содержимого базовых таблиц недоступно.

Соответствие ролей и допустимых операций задаётся тремя вариантами использования.

- 1) просмотр статистики — получение сведений о матчах, игроках и командах, агрегированных показателях и рейтингах; доступно всем ролям;

- 2) расчёт метрик — вычисление продвинутых показателей (PER, BPM, eFG%, TS%, USG%) на основе первичных данных; доступно аналитику и администратору;
- 3) управление данными — загрузка данных о матчах, обновление справочников, изменение схемы; доступно только администратору.

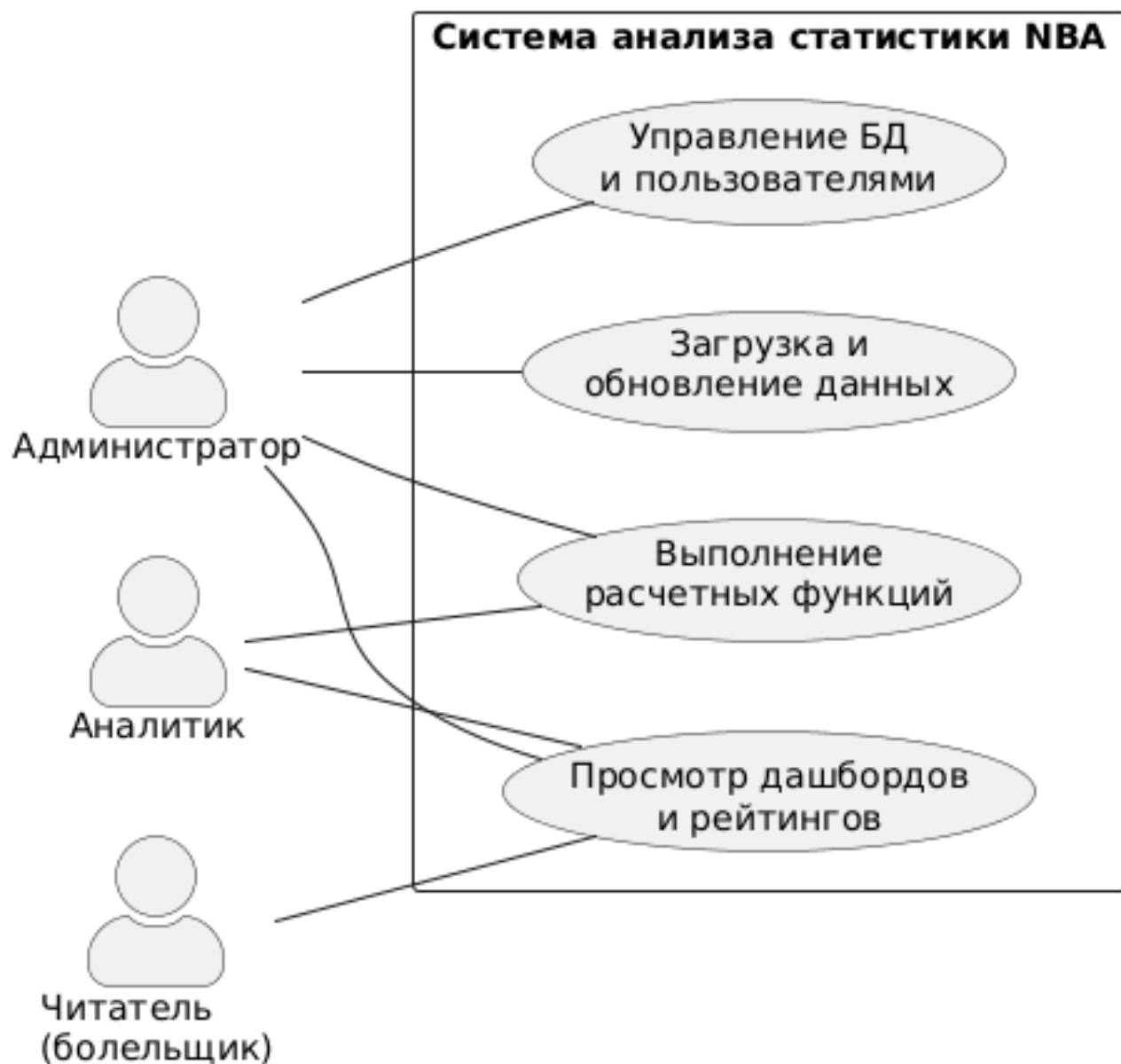


Рисунок 1.2 — Диаграмма вариантов использования

1.6 Выбор модели данных

Для выбора подходящей модели хранения данных проведём сравнение трёх основных парадигм, опираясь на принципы проектирования дата-интенсивных приложений [12].

Документно-ориентированные СУБД (MongoDB, Couchbase). Как отмечают П. Садаладж и М. Фаулер [13], такие базы данных оптимальны для хранения агрегированных JSON-документов без жёсткой схемы (schema-less). Сырая статистика матча (Box Score) естественно ложится в документную модель. Однако данный подход существенно проигрывает в произво-

дительности при многомерных агрегациях по нескольким сезонам и не поддерживает декларативную валидацию бизнес-правил на уровне хранилища.

Колоночные СУБД (ClickHouse и аналогичные OLAP-системы) ориентированы на аналитику больших объёмов данных: хранение по столбцам многократно ускоряет агрегирующие запросы (GROUP BY, SUM, AVG) [14]. Однако, как указывается в теории баз данных, OLAP-системы не обеспечивают полноценных транзакций ACID при смешанных нагрузках, что критично для согласованности данных матчей, и существенно усложняют реализацию JOIN-запросов между справочниками и таблицами фактов.

Реляционные СУБД (PostgreSQL, MySQL) гарантируют строгую согласованность данных (ACID) и обладают мощным оптимизатором SQL-запросов [15]. В отличие от NoSQL, реляционная модель (schema-on-write) позволяет встроить валидацию бизнес-логики непосредственно в СУБД через ограничения (CHECK) и триггеры, что обеспечивает достоверность спортивной статистики.

Таблица 1.2 — Сравнительная характеристика моделей хранения данных

Критерий	Документная СУБД	Колоночная СУБД	Реляционная СУБД
Контроль над схемой	Гибкая (schema-less)	Жёсткая по столбцам	Строгая (schema-on-write)
Транзакционность (ACID)	Частичная	Не поддерживается при OLTP-нагрузке	Полная
Сложные JOIN-запросы	Затруднены	Затруднены	Встроенная поддержка
Агрегации по сезонам	Средняя (MapReduce)	Высокая (по столбцам)	Высокая (оптимизатор SQL)
Валидация бизнес-логики	На уровне приложения	На уровне приложения	Встроенная (триггеры, CHECK)
Соответствие структуре матча	Идеальное (JSON)	Требует денормализации	Требует декомпозиции

Колоночные СУБД демонстрируют преимущество преимущественно при объёмах от десятков миллионов строк и строго однородной аналитической нагрузке класса OLAP. Рассматриваемая система предполагает смешанный профиль использования. Процедура обновления сезонной статистики выполняет ресурсоёмкие транзакционные операции слияния для сотен записей одновременно. Кроме того, необходимость ретроспективной корректировки данных из-за ошибок в официальных протоколах матчей требует гарантированно атомарного обновления сразу нескольких связанных таблиц.

При целевом объёме таблицы фактов порядка 200 000 строк классическая реляционная база данных способна обрабатывать агрегирующие запросы без существенных задержек. Для наиболее востребованных аналитических выборок, таких как списки лидеров и сводные турнирные таблицы, применяется кэширование в Redis [16], что полностью нивелирует отставание в скорости чтения от специализированных аналитических решений. Использование колоночной СУБД при таком масштабе неизбежно привело бы к усложнению системы за счёт выделения отдельного транзакционного хранилища для операций записи. Подобная архитектурная избы-

точность не даст измеримого выигрыша в производительности.

Таким образом, с учётом требований к транзакционной целостности и предсказуемого объёма данных, оптимальным выбором является реляционная модель [3, 15]. В качестве основной СУБД выбран PostgreSQL 15 [17].

Вывод

Сформулированы функциональные и нефункциональные требования и ограничения; определён перечень метрик и правила их вычисления. Выделены восемь сущностей, установлены связи «многие-ко-многим» через таблицы фактов, описаны три пользовательские роли. По результатам сравнения моделей хранения данных выбрана реляционная модель (PostgreSQL 15): она единственная обеспечивает полный ACID, произвольные JOIN-запросы и встроенную валидацию бизнес-логики. Сравнительный анализ существующих систем (NBA.com Stats, Basketball-Reference, ESPN Stats) подтвердил отсутствие у них произвольных аналитических запросов, управляемого ролевого доступа и воспроизводимого локального хранения данных, что обуславливает целесообразность разработки специализированной базы данных. Диаграмма вариантов использования и ER-диаграмма приведены на рисунках 1.2 и 1.1.

2 Конструкторский раздел

2.1 Логическое проектирование базы данных

Инфологическая модель трансформирована в реляционную схему из восьми таблиц. Справочник `seasons` использует суррогатный ключ `season_id` и метку `label` (формат «ГГГГ-ГГ»). Сборка фактов осуществляется в таблице `game_player_stats`, сезонные агрегаты хранятся в таблице `player_season_stats` в виде денормализованной витрины данных для минимизации времени отклика. Сводная схема связей между таблицами приведена на рисунке 2.1.

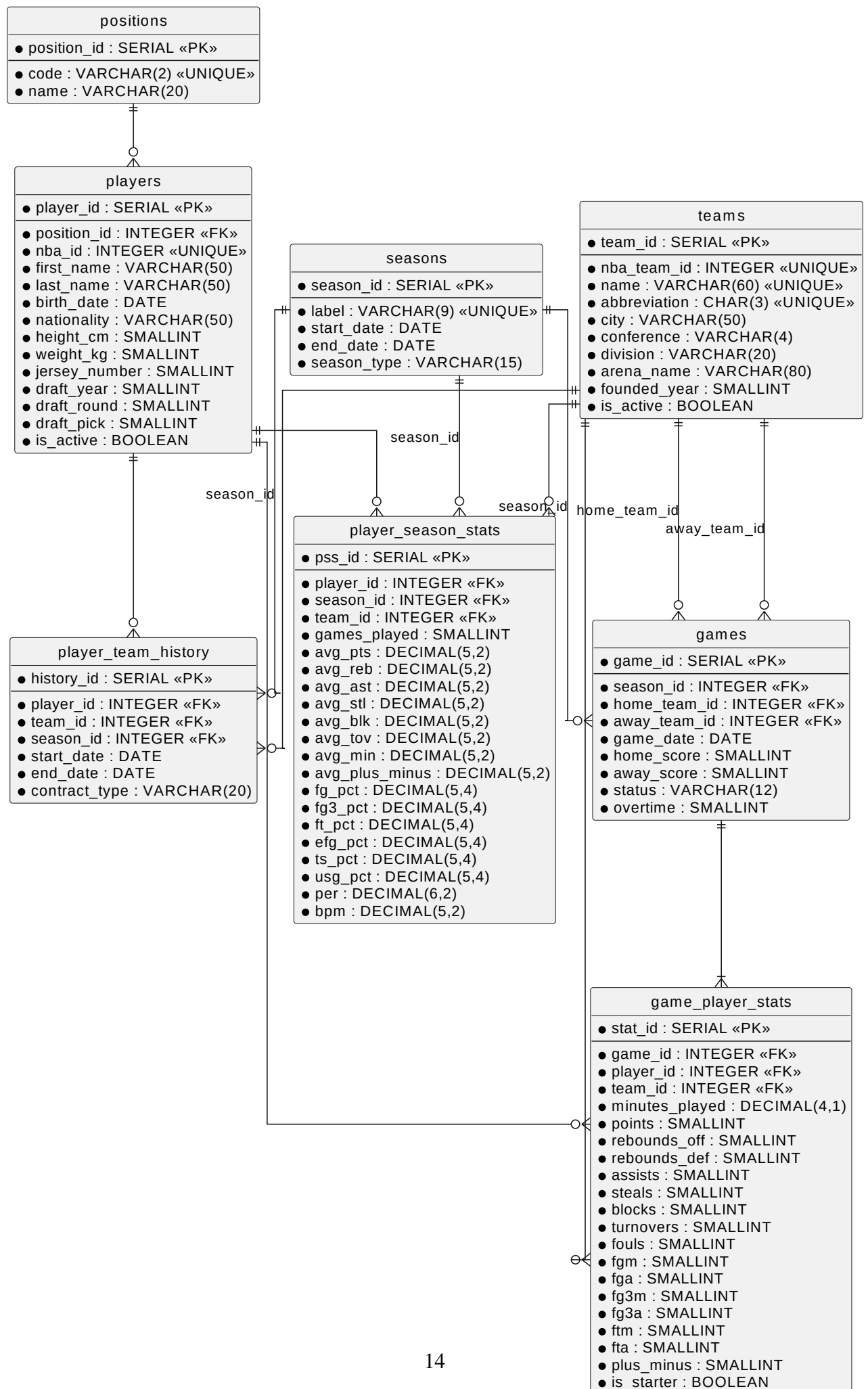


Таблица 2.1 — Таблица positions (справочник позиций)

Атрибут	Тип	Ключ / назначение
position_id	SERIAL	PK
code	VARCHAR(2)	UNIQUE; код PG, SG, SF, PF, C
name	VARCHAR(20)	полное название позиции

Таблица 2.2 — Таблица seasons (сезоны)

Атрибут	Тип	Ключ / назначение
season_id	SERIAL	PK
label	VARCHAR(9)	UNIQUE; метка сезона (напр. 2023-24)
start_date	DATE	дата начала сезона
end_date	DATE	дата окончания сезона
season_type	VARCHAR(15)	Regular, Playoff, Preseason

Таблица 2.3 — Таблица teams (команды)

Атрибут	Тип	Ключ / назначение
team_id	SERIAL	PK
nba_team_id	INTEGER	UNIQUE; id на NBA.com
name	VARCHAR(60)	UNIQUE; полное название
abbreviation	CHAR(3)	UNIQUE; аббревиатура
city	VARCHAR(50)	город базирования
conference	VARCHAR(4)	CHECK IN ('East','West')
division	VARCHAR(20)	дивизион NBA
arena_name	VARCHAR(80)	домашняя арена
founded_year	SMALLINT	год основания
is_active	BOOLEAN	признак активной команды

Таблица 2.4 — Таблица players (игроки)

Атрибут	Тип	Ключ / назначение
player_id	SERIAL	PK
nba_id	INTEGER	UNIQUE; id на NBA.com
first_name	VARCHAR(50)	имя
last_name	VARCHAR(50)	фамилия
birth_date	DATE	дата рождения
nationality	VARCHAR(50)	гражданство
height_cm	SMALLINT	рост (см)
weight_kg	SMALLINT	вес (кг)
position_id	INTEGER	FK → positions
jersey_number	SMALLINT	номер на майке (0–99)
is_active	BOOLEAN	действующий игрок
draft_year	SMALLINT	год драфта
draft_round	SMALLINT	раунд драфта (1 или 2)
draft_pick	SMALLINT	номер выбора на драфте

Таблица 2.5 — Таблица games (матчи)

Атрибут	Тип	Ключ / назначение
game_id	SERIAL	PK
season_id	INTEGER	FK → seasons
home_team_id	INTEGER	FK → teams; команда-хозяин
away_team_id	INTEGER	FK → teams; команда-гость
game_date	DATE	дата матча
home_score	SMALLINT	счёт хозяев
away_score	SMALLINT	счёт гостей
status	VARCHAR(12)	Finished, Scheduled, Postponed
overtime	SMALLINT	число овертаймов (0–6)

Таблица 2.6 — Таблица game_player_stats (факты матча)

Атрибут	Тип	Ключ / назначение
stat_id	SERIAL	PK
game_id	INTEGER	FK → games, ON DELETE CASCADE
player_id	INTEGER	FK → players, ON DELETE RESTRICT
team_id	INTEGER	FK → teams, ON DELETE RESTRICT
minutes_played	DECIMAL(4,1)	минуты на площадке
points	SMALLINT	очки (CHECK ≥ 0)
rebounds_off, rebounds_def	SMALLINT	подборы в нападении и защите
assists, steals, blocks	SMALLINT	передачи, перехваты, блоки
turnovers, fouls	SMALLINT	потери и фолы
fgm, fga, fg3m, fg3a, ftm, fta	SMALLINT	попадания и попытки бросков
plus_minus	SMALLINT	показатель плюс-минус
is_starter	BOOLEAN	признак стартового состава
UNIQUE	—	(game_id, player_id) — гарантирует единственность записи игрока в матче

В таблице game_player_stats пара полей game_id и player_id закреплена ограничением UNIQUE. Это исключает дублирование строк статистики одного игрока в одном матче.

Таблица 2.7 — Таблица player_season_stats (агрегаты за сезон)

Атрибут	Тип	Ключ / назначение
pss_id	SERIAL	PK
player_id	INTEGER	FK → players; часть UNIQUE
season_id	INTEGER	FK → seasons; часть UNIQUE
team_id	INTEGER	FK → teams; часть UNIQUE
games_played	SMALLINT	число игр в сезоне
avg_pts, avg_reb, avg_ast	DECIMAL(5,2)	средние за матч (очки, подборы, передачи)
avg_stl, avg_blk, avg_tov, avg_min	DECIMAL(5,2)	перехваты, блоки, потери, минуты
fg_pct, fg3_pct, ft_pct	DECIMAL(5,4)	проценты попаданий (дробь ∈ [0, 1])
avg_plus_minus	DECIMAL(5,2)	средний плюс-минус
efg_pct, ts_pct, usg_pct	DECIMAL(5,4)	eFG%, TS%, USG (доля 0–1; на UI × 100)
per	DECIMAL(6,2)	PER
bpm	DECIMAL(5,2)	BPM

Таблица 2.8 — Таблица `player_team_history` (история выступлений)

Атрибут	Тип	Ключ / назначение
<code>history_id</code>	SERIAL	PK
<code>player_id</code>	INTEGER	FK → <code>players</code> ; часть UNIQUE
<code>team_id</code>	INTEGER	FK → <code>teams</code>
<code>season_id</code>	INTEGER	FK → <code>seasons</code>
<code>start_date</code>	DATE	дата начала выступления
<code>end_date</code>	DATE	дата окончания (NULL — по сей день)
<code>contract_type</code>	VARCHAR(20)	Standard, Two-Way, 10-Day и др.

2.1.1 Соответствие нормальным формам

Схема нормализована до третьей нормальной формы (3НФ). Ниже для всех восьми таблиц проверяется выполнение первой (1НФ), второй (2НФ) и третьей (3НФ) нормальных форм [15].

Первая нормальная форма (1НФ). В каждом отношении атрибуты атомарны: повторяющихся групп и многозначных полей нет. Статистика игрока в матче хранится отдельной строкой в `game_player_stats`, а не списком показателей в одной ячейке; справочники (`positions`, `teams`) и сущности (`players`, `games`) разнесены по отдельным таблицам.

Вторая нормальная форма (2НФ). Отношение в 2НФ, если оно в 1НФ и каждый неключевой атрибут полностью зависит от всего составного ключа. У таблиц `positions`, `seasons`, `teams`, `players`, `games` первичный ключ простой (`*_id`), поэтому частичных зависимостей быть не может. В `game_player_stats` показатели зависят от пары `game_id` и `player_id`, а не от каждого поля по отдельности. Аналогично в `player_season_stats` агрегаты зависят от составного ключа: `player_id`, `season_id`, `team_id`. В `player_team_history` атрибут `contract_type` относится к конкретному периоду игрока в команде и зависит от составного ключа записи.

Третья нормальная форма (3НФ). Отношение в 3НФ, если оно в 2НФ и для каждой нетривиальной зависимости $X \rightarrow A$ либо X — суперключ, либо A входит в потенциальный ключ. В `teams` поля `city` и `conference` описывают команду напрямую и зависят только от `team_id`. В `players` ФИО и физические параметры зависят только от `player_id`, а не от `team_id` (команда задаётся через `player_team_history`). Совпадение данных `player_season_stats` с агрегатами из `game_player_stats` — избыточность между таблицами (витрина для быстрых выборов), а не нарушение 3НФ внутри одного отношения.

Таким образом, все таблицы схемы находятся в 1НФ, 2НФ и 3НФ.

2.1.2 Проектирование представлений

Для обеспечения доступа к данным без прямого обращения к базовым таблицам спроектированы четыре представления:

- 1) `v_player_rankings` — объединяет `player_season_stats` и `players`; предоставляет

- сводный рейтинг игроков за сезон по очкам, подборам, передачам и расчётным метрикам;
- 2) `v_team_stats` — агрегирует данные `game_player_stats` в разрезе команды и сезона; реализует функциональное требование расчёта командной статистики без отдельной таблицы фактов;
 - 3) `v_team_standings` — вычисляет турнирное положение команд (число побед и поражений, процент побед) на основе завершённых матчей;
 - 4) `v_top_players` — краткая сводка лидеров по PER для роли reader (подмножество `v_player_rankings`).

2.1.3 Проектирование индексов

Для обеспечения производительности аналитических запросов в соответствии с нефункциональными требованиями (раздел 1.3.2) спроектированы следующие индексы [17]:

- 1) `idx_gps_player_id` ON `game_player_stats(player_id)` — ускоряет выборку всех выступлений игрока для агрегации сезонной статистики;
- 2) `idx_gps_team_game` ON `game_player_stats(team_id, game_id)` — поддерживает агрегацию командной статистики в представлении `v_team_stats`;
- 3) `idx_games_season_id` ON `games(season_id)` — ускоряет фильтрацию матчей по сезону;
- 4) `idx_games_date` ON `games(game_date)` — обеспечивает диапазонную фильтрацию по дате проведения матча;
- 5) `idx_pss_season` ON `player_season_stats(season_id)` — ускоряет выборку рейтингов за конкретный сезон;
- 6) `idx_players_name` ON `players(last_name, first_name)` — поддерживает поиск игрока по фамилии.

Ограничение UNIQUE на `(game_id, player_id)` в таблице `game_player_stats` автоматически создаёт B-tree индекс, покрывающий запросы поиска по паре «матч – игрок». Все индексы реализованы как B-tree — стандартная структура PostgreSQL, оптимальная для равенства и диапазонных предикатов в аналитических запросах.

2.2 Проектирование ограничений целостности

Помимо стандартных ограничений первичных (PK) и внешних (FK) ключей, в СУБД реализована логика защиты бизнес-правил.

Внешние ключи и каскадное удаление. При удалении матча из `games` связанные строки `game_player_stats` удаляются каскадно (ON DELETE CASCADE). Для справочников игроков и команд применяется стратегия ON DELETE RESTRICT: физическое удаление запрещено во избежание потери исторической статистики; вместо этого используется логическое удаление посредством флага `is_active`.

ЧЕКС-ограничения. Обеспечивают согласованность бросков: $fg3m \leq fgm$, $fgm \leq fga$;

диапазон фолов от 0 до 6. В games поля home_team_id и away_team_id различаются. В player_team_history выполняется $\text{start_date} \leq \text{end_date}$.

Бизнес-триггеры. На таблице game_player_stats реализованы два триггера. Триггер trg_check_team_score (AFTER INSERT OR UPDATE, FOR EACH ROW) валидирует равенство суммы очков игроков ($\sum \text{points}$) командному счёту (home_score или away_score) для завершённых матчей. Триггер after_game_player_stats_insert (AFTER INSERT, FOR EACH STATEMENT) вызывает update_season_stats для пересчёта витрины. Ограничение по счёту применимо к полностью обработанным данным: игроки, не принявшие участие в матче, в таблицу фактов game_player_stats они не вносятся; загружаемые источники должны обеспечивать атрибуцию всех очков конкретным игрокам.

2.3 Проектирование алгоритмов обработки данных

2.3.1 Алгоритмы расчёта метрик игроков

Функции расчёта метрик выполняются на стороне СУБД. При проектировании алгоритма учтено деление на ноль: если знаменатель равен нулю, функция возвращает NULL, что позволяет агрегатным функциям AVG() корректно исключить таких игроков из расчёта командной и лиговой статистики без внесения нулевых искажений.

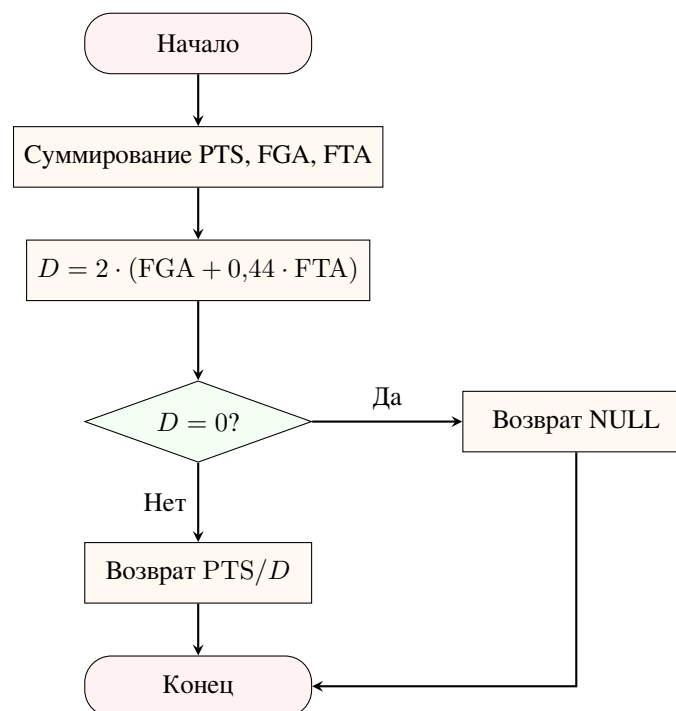


Рисунок 2.2 — Алгоритм функции calculate_pts

2.3.2 Алгоритм агрегирования командной статистики

Командная статистика не хранится в отдельной таблице: в соответствии с проектным решением она вычисляется динамически представлением v_team_stats. Для каждой пары «ко-

манда – сезон» представление суммирует показатели всех игроков из `game_player_stats` по матчам, в которых данная команда выступала в качестве хозяев или гостей, и делит накопленные суммы на число матчей. Принадлежность строки статистики к хозяевам или гостям определяется через JOIN с таблицей `games` по полям `home_team_id` и `away_team_id`. Результирующий набор содержит средние очки, подборки, передачи, потери и эффективные показатели команды за матч.

2.3.3 Алгоритм массового обновления агрегатов

Процедура `update_season_stats` пересчитывает агрегаты для указанного сезона. Алгоритм спроектирован с отказом от курсоров и построчного обхода. Применяется множественная операция с использованием конструкции `INSERT ... ON CONFLICT DO UPDATE`. Это позволяет СУБД задействовать внутренний оптимизатор и многократно сократить время пересчёта. Атомарность операции UPSERT на уровне СУБД предотвращает состояния гонки при параллельном пересчёте.

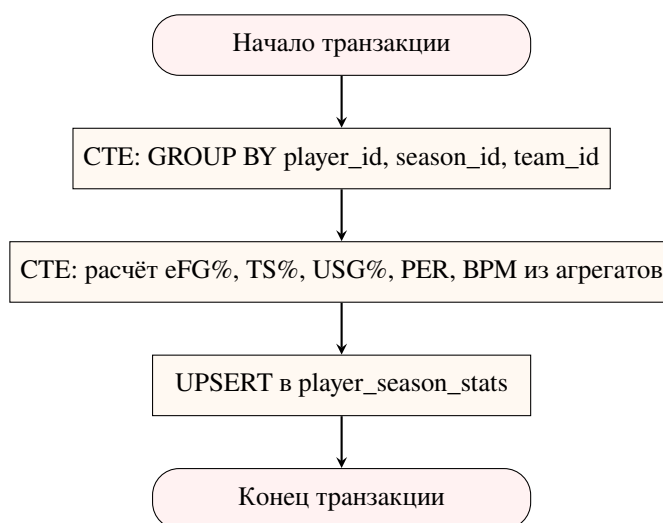


Рисунок 2.3 — Алгоритм обновления агрегатов

2.4 Проектирование ролевой модели доступа

В базе данных определены три роли: `db_admin`, `analyst` и `reader`. Администратору выданы полные права на таблицы, последовательности и функции схемы. Аналитику доступны выборка из таблиц и выполнение расчётных функций. Читатель работает исключительно через представления без прямого доступа к базовым таблицам. Создание и изменение структуры объектов базы данных выполняются от имени выделенного пользователя-владельца схемы при развёртывании. Ограничение доступа роли `reader` к полному представлению `v_player_rankings` и выдача прав только на `v_top_players` обусловлены бизнес-логикой: публичным пользователям доступна лишь краткая сводка лидеров, в то время как полные аналитические выгрузки предназначены только для аналитиков.

Таблица 2.9 — Матрица прав доступа к объектам базы данных

Объект	db_admin	analyst	reader
positions, seasons, teams, players	CRUD	SELECT	—
games, game_player_stats	CRUD	SELECT	—
player_season_stats, player_team_history	CRUD	SELECT	—
Функции calculate_*	EXECUTE	EXECUTE	—
v_team_stats, v_team_standings, v_top_players	—	SELECT	SELECT
v_player_rankings	SELECT	SELECT	—

2.5 Проектирование программного обеспечения и кэширования

2.5.1 Архитектура приложения

Программное обеспечение реализует трёхзвенную архитектуру. Уровень представления — веб-клиент, формирующий HTTP-запросы к серверу. Уровень приложения — REST API-сервер, выполняющий маршрутизацию запросов, проверку авторизации и вызов процедур СУБД. Уровень данных — реляционная СУБД и in-memory кэш, взаимодействие которых описано в подразделе 2.5.3.

API организован в пять групп маршрутов (players, teams, stats, league, admin), соответствующих вариантам использования из раздела 1.5.2. Итого спроектировано 21 бизнес-маршрут.

2.5.2 Спецификация REST API

Сезон в параметрах запроса передаётся целочисленным season_id (внешний ключ таблицы seasons); метка label (например, 2023–24) возвращается в ответах API для отображения в интерфейсе. Список сезонов клиент получает через GET /league/seasons.

Таблица 2.10 — Маршруты группы /players

Метод	Путь	Параметры	Вариант исп.
GET	/players	season_id (обяз.), position, team_id, search, limit, offset	Просмотр статистики
GET	/players/{player_id}	—	Просмотр статистики
GET	/players/{player_id}/stats	season_id (опц.)	Просмотр статистики
GET	/players/{player_id}/gamelog	season_id (обяз.)	Просмотр статистики
GET	/players/{player_id}/career	—	Просмотр статистики
GET	/players/{player_id}/teams	—	Просмотр статистики

Таблица 2.11 — Маршруты группы /teams

Метод	Путь	Параметры	Вариант исп.
GET	/teams	—	Просмотр статистики
GET	/teams/standings	season_id (обяз.)	Просмотр статистики
GET	/teams/{team_id}	—	Просмотр статистики
GET	/teams/{team_id}/roster	season_id (обяз.)	Просмотр статистики
GET	/teams/{team_id}/games	season_id (опц.)	Просмотр статистики

Таблица 2.12 — Маршруты группы /stats

Метод	Путь	Параметры	Вариант исп.
GET	/stats/leaders	metric, season_id (обяз.), team_id, position, min_games, limit	Просмотр статистики
GET	/stats/advanced	season_id (обяз.)	Расчёт метрик
GET	/stats/scatter	season_id (обяз.)	Расчёт метрик
GET	/stats/boxscore/{game_id}	—	Просмотр статистики
GET	/stats/compare/players	p1_id, p2_id, season_id (обяз.)	Просмотр статистики

Параметр `metric` в маршруте `/stats/leaders` принимает одно из значений: `avg_pts`, `avg_reb`, `avg_ast`, `avg_stl`, `avg_blk`, `per`, `ts_pct`, `efg_pct`, `bpm` и `usg_pct`.

Таблица 2.13 — Маршруты группы /league

Метод	Путь	Параметры	Вариант исп.
GET	/league/seasons	—	Просмотр статистики
GET	/league/dashboard	season_id (обяз.)	Просмотр статистики
GET	/league/trends	—	Просмотр статистики
GET	/league/search	q (обяз., мин. 2 символа)	Просмотр статистики

Маршрут /league/seasons возвращает перечень доступных сезонов. Ответ содержит поля season_id, label, season_type, start_date и end_date; клиент использует его для формирования элементов выбора сезона.

Таблица 2.14 — Маршруты группы /admin

Метод	Путь	Параметры	Вариант исп.
POST	/admin/refresh/{season_id}	заголовок X-API-Key	Управление данными

Разграничение доступа на уровне API реализовано для группы /admin: маршрут проверяет наличие корректного значения в заголовке X-API-Key. Разграничение ролей analyst и reader обеспечивается средствами СУБД в соответствии с матрицей прав (таблица 2.9): API-сервер устанавливает соединение с базой данных от имени пользователя с соответствующим набором привилегий.

2.5.3 Кэширование результатов запросов

Для обеспечения высокой пропускной способности спроектирован паттерн Cache-Aside с использованием in-memory хранилища и API-сервера.

Алгоритм: приложению поступает REST-запрос. Сервер проверяет наличие ключа в кэше. При попадании данные возвращаются из кэша. При промахе сервер запрашивает реляционную БД, сериализует ответ, сохраняет его в кэш с заданным временем жизни и возвращает клиенту.

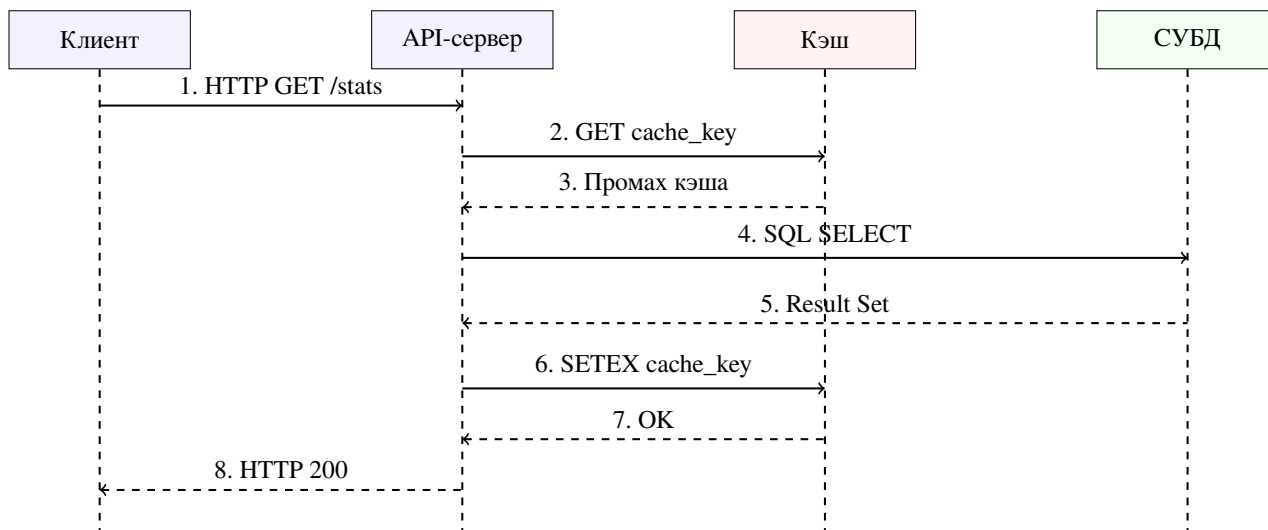


Рисунок 2.4 — UML-диаграмма последовательности при паттерне Cache-Aside

Инвалидация кэша выполняется по префиксам конкретных матчей и турнирных таблиц только для изменённого сезона, а не полным сбросом кэша: это предотвращает одновременную инвалидацию всех ключей и снижает риск пиковой нагрузки на СУБД при массовом промахе кэша — эффекта «шторма кэша», который возникает, когда множество параллельных запросов одновременно обнаруживают истёкший ключ и нагружают базу данных.

Вывод

Спроектирована реляционная схема из 8 таблиц, 4 представлений и 6 ключевых индексов; показано соответствие всех таблиц 1НФ, 2НФ и 3НФ. Бизнес-логика метрик реализована SQL-процедурами на стороне СУБД; ограничения целостности закреплены на уровне CHECK-ограничений и триггера. Определена ролевая модель из трёх ролей с разграничением прав на уровне таблиц, функций и представлений. Спроектировано REST API из 21 маршрута, организованных в пять групп и покрывающих все три варианта использования системы. Для снижения задержки аналитических выборок спроектировано кэширование по паттерну Cache-Aside с префиксной инвалидацией.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. NBA Stats [Электронный ресурс]. — Официальная статистика Национальной баскетбольной ассоциации. — Режим доступа: <https://www.nba.com/stats> (дата обращения: 29.03.2026).
2. Kubatko, J. et al. A Starting Point for Analyzing Basketball Statistics // Journal of Quantitative Analysis in Sports. — 2007. — Vol. 3, № 3. — Article 1.
3. Коннолли, Т. Базы данных. Проектирование, реализация и сопровождение. Теория и практика / Т. Коннолли, К. Бегг ; пер. с англ. — 3-е изд. — М. : Вильямс, 2003. — 1440 с.
4. Basketball-Reference.com [Электронный ресурс]. — Sports Reference LLC. Справочник статистики NBA и исторические таблицы. — Режим доступа: <https://www.basketball-reference.com/> (дата обращения: 29.03.2026).
5. ESPN NBA [Электронный ресурс]. — Новости и обзоры НБА. — Режим доступа: <https://www.espn.com/nba/> (дата обращения: 29.03.2026).
6. Hollinger, J. Pro Basketball Forecast. — Washington, D.C.: Potomac Books, 2005. — 432 p.
7. Player Efficiency Rating [Электронный ресурс] // Basketball-Reference.com. — Описание показателя PER (упрощенная методология). — Режим доступа: <https://www.basketball-reference.com/about/per.html> (дата обращения: 29.03.2026).
8. Myers, D. About Box Plus/Minus (BPM) 2.0 [Электронный ресурс] // Basketball-Reference.com. — Режим доступа: <https://www.basketball-reference.com/about/bpm2.html> (дата обращения: 29.03.2026).
9. Glossary — NBA.com : Stats [Электронный ресурс]. — Термины и определения показателей. — Режим доступа: <https://www.nba.com/stats/help/glossary> (дата обращения: 29.03.2026).
10. Oliver, D. Basketball on Paper: Rules and Tools for Performance Analysis. — Washington, D.C.: Potomac Books, 2004. — 378 p.
11. Chen, P. P. The Entity-Relationship Model — Toward a Unified View of Data / P. P. Chen // ACM Transactions on Database Systems. — 1976. — Vol. 1, № 1. — P. 9–36.
12. Клеппман, М. Высоконагруженные приложения. Программирование, масштабирование, поддержка / М. Клеппман. — СПб.: Питер, 2018. — 640 с.

13. Садаладж, П. Дж. NoSQL: новая методология разработки нереляционных баз данных / П. Дж. Садаладж, М. Фаулер. — М.: Вильямс, 2013. — 192 с.
14. ClickHouse Documentation [Электронный ресурс]. — Официальная документация СУБД ClickHouse. — Режим доступа: <https://clickhouse.com/docs/> (дата обращения: 29.03.2026).
15. Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт ; пер. с англ. — 8-е изд. — М. : Вильямс, 2006. — 1328 с.
16. Redis Documentation [Электронный ресурс]. — Документация in-memory хранилища Redis. — Режим доступа: <https://redis.io/docs/> (дата обращения: 29.03.2026).
17. PostgreSQL 15 Documentation [Электронный ресурс]. — Официальная документация СУБД PostgreSQL. — Режим доступа: <https://www.postgresql.org/docs/15/> (дата обращения: 29.03.2026).